

AI Agent Execution Platform

Problem Statement

AI agents are increasingly being used to perform real-world tasks rather than simply generate text.

For example, an accounting assistant could receive a request such as:

"Find all overdue invoices, identify the customers who need a reminder, prepare reminder emails, and send them after I approve."

Completing this request may require the agent to perform multiple actions, use external tools, maintain state between steps, handle failures, and potentially wait for user input before continuing.

Your task is to design and build the **backend for an AI Agent Execution Platform** that can reliably execute these kinds of multi-step tasks.

The platform should allow users to define AI agents, provide them with a set of tools, and submit tasks for execution.

Goal

Build a system that can:

- Create and manage AI agents.
- Give agents access to a configurable set of tools.
- Accept tasks from users and execute them using an LLM.
- Support multi-step agent execution.
- Maintain the state and history of an execution.
- Execute tasks asynchronously.
- Handle failures and retries gracefully.
- Resume an execution when possible after a failure.
- Support actions that require human approval.
- Support multiple users and concurrent executions.

The system should be designed with production use in mind.

Example Use Case

Consider an accounting assistant with access to tools such as:

- Retrieve invoices
- Retrieve customer information
- Create an email draft
- Send an email

A user might ask:

"Find all overdue invoices above ₹50,000 and prepare reminder emails for the respective customers."

The agent should be able to determine the appropriate sequence of actions, use the available tools, process their results, and eventually provide the user with the outcome.

For a different request:

"Find overdue invoices and send reminder emails to the customers."

The system may need to pause before sending the emails and obtain approval from the user.

The exact implementation and architecture are up to you.

Key Requirements

Agent Management

Users should be able to create and manage agents.

An agent should have appropriate configuration such as its instructions, available capabilities/tools, and model configuration.

Different agents should be able to have different capabilities.

Task Execution

Users should be able to submit a task to an agent.

A task may require multiple LLM interactions and multiple tool calls before it is completed.

The system should provide a way for users to track the progress and final outcome of their executions.

Tool Execution

Agents should be able to use tools to perform actions.

Tools may represent internal services, external APIs, databases, or other operations.

The platform should provide a mechanism for controlling which tools are available to a particular agent.

At least a few representative tools should be implemented to demonstrate the execution flow.

Asynchronous Processing

Agent executions may take significant time and should not require the initial user request to remain open until completion.

The system should be capable of handling multiple executions concurrently.

Reliability

The platform should account for real-world failures such as:

- LLM failures
- Tool/API failures
- Timeouts
- Temporary infrastructure failures
- Worker/process failures
- Invalid tool requests
- Duplicate requests
- Unexpected LLM behaviour

The system should avoid losing execution state when failures occur.

Execution Recovery

An agent may complete several steps successfully before the process executing it fails.

The system should be designed so that an execution can recover from such failures without unnecessarily repeating completed work.

Explain the approach and trade-offs in your design.

Human Approval

Some actions may have real-world consequences and should require explicit user approval.

For example, an agent may be allowed to draft an email automatically but require approval before actually sending it.

The system should support pausing an execution for approval and continuing it once the user responds.

Execution History

Users should be able to understand what happened during an execution.

The system should retain sufficient information to answer questions such as:

- What task was requested?
 - Which agent executed it?
 - Which steps were performed?
 - Which tools were called?
 - What failed?
 - What was retried?
 - What is the current status?
 - What was the final result?
-

Production Considerations

Think beyond the happy path.

Your solution should consider:

- Concurrency
- Idempotency
- Retries
- Timeouts
- State management
- Failure recovery
- Security
- Isolation between users/tenants
- Observability
- Scalability
- Cost and latency of LLM calls

You do not need to build every possible production feature, but your architecture and implementation should demonstrate how you would approach these concerns.

Technology

You are free to choose the technology stack and architecture.

You may use any reasonable:

- Programming language
- Web framework
- Database

- Queue/message broker
- LLM provider
- Infrastructure approach

We care more about the engineering decisions and quality of the solution than the specific technologies chosen.

Deliverables

Please provide:

1. **Source code** in a GitHub repository.
 2. **README** with setup instructions and an explanation of your approach.
 3. **Architecture/design documentation** explaining the major components and decisions.
 4. **Tests** covering important parts of the system.
 5. Any additional documentation or diagrams you feel are useful.
-

Evaluation

The assignment will be evaluated primarily on:

Engineering Quality

- Code quality and maintainability
- API and data modelling
- Testing
- Error handling
- Clear separation of responsibilities

System Design

- Architecture
- Scalability
- Concurrency
- State management
- Reliability
- Failure recovery
- Trade-offs

AI/Agent Engineering

- Agent execution design
- LLM integration
- Tool usage
- Handling unexpected LLM behaviour

- Context and execution state
- Guardrails

Production Readiness

We are particularly interested in how the system behaves when things go wrong.

A solution that works only for the happy path is not sufficient.

Bonus

Additional credit may be given for thoughtful implementation of areas such as:

- Streaming execution updates
- Parallel tool execution
- Tool permissions
- Rate limiting
- LLM provider fallback
- Token/cost management
- Advanced observability
- Distributed tracing
- Dead-letter handling
- Execution cancellation
- Multi-tenant isolation
- Load testing

These are optional. Candidates should prioritize a **reliable and well-designed core system** over implementing a large number of features.

Expected Effort

Approximately **6–8 hours**.

You are free to use AI coding assistants and other AI tools while completing the assignment.

Please mention the AI tools used and how they contributed to the implementation in the README.